

# Construirea și calibrarea unui pipeline hibrid de RAG

Tema 2 MDAD (Mineritul Datelor și Analiza Datelor) 2026  
v1.0

## 1. Descriere generală

În această temă veți construi end-to-end un pipeline de retrieval-augmented generation (RAG), veți evalua trei strategii diferite de retrieval pe trei seturi de date din benchmark-ul BEIR și veți calibra un hiperparametru al sistemului. Activitatea reflectă modul în care sistemele RAG sunt construite și măsurate în practică.

Pipeline-ul pe care îl veți construi combină retrieval sparse (BM25), dens (vectorial) și hibrid peste o bază de date vectorială, iar contextul regăsit va fi trimis unui LLM mic rulat local, care va genera răspunsuri cu citări.

## 2. Concepte explorate

Tema integrează majoritatea conceptelor discutate despre Text Mining:

- retrieval sparse cu BM25
- retrieval dens cu un bi-encoder peste un index format cu Approximate Nearest Neighbour
- căutare hibridă cu fuziune convexă a scorurilor
- strategii de chunking
- RAG end-to-end cu un LLM local
- evaluarea standard a sistemelor de regăsire a informațiilor (IR) prin Recall@k, MRR și nDCG@k

## 3. Tehnologii utilizate

Toate componentele sunt gratuite și rulează local pe laptop. Baza de date vectorială va fi **Weaviate**, implementată și rulată prin Docker Compose în mod BYO-vectors (BringYourOwn-vectors, adică generați voi vectorii de embeddings), fără un modul de vectorizare configurat — astfel totul rămâne offline, fără chei API. LLM-ul va rula prin **Ollama**, iar modelul recomandat default este Gemma 2 2B (încapă confortabil în 8 GB RAM); în cazul laptop-urilor cu cel puțin 16 GB puteți opta pentru Llama 3.2 3B pentru răspunsuri mai bune. Pentru embeddings se va folosi **all-MiniLM-L6-v2** din pachetul **sentence-transformers**.

Veți avea nevoie de Python 3.10 sau mai recent, plus pachetele **beir**, **weaviate-client**, **sentence-transformers** și **ollama**. Hardware-ul minim este de 8 GB RAM (16 GB recomandat) și aproximativ 10 GB spațiu liber pe disc. CPU-only este suficient pentru a rula toată tema, nefiind nevoie de GPU (deși dacă aveți unul cu suficient VRAM îl puteți folosi pentru accelerare).

## 4. Seturi de date

Veți folosi trei seturi de date din benchmark-ul BEIR, toate accesibile prin pachetul Python `beir`. Acestea sunt alese pentru a expune comportamente diferite ale retrieverelor pe domenii diferite.

| Set de date     | Dimensiune      | Domeniu                  |
|-----------------|-----------------|--------------------------|
| <b>NFCorpus</b> | ~3.6K documente | Medical / nutriție       |
| <b>SciFact</b>  | ~5K documente   | Fact-checking științific |
| <b>FiQA</b>     | ~57K documente  | Q&A financiar            |

Fiecare set vine cu query-uri și teste de relevanță (qrels) preprocesate. Corpusul combinat este de aproximativ 65K documente, o dimensiune gestionabilă pe un laptop mediu. Încărcarea datelor presupune practic trei linii per set de date:

```
from beir import util
from beir.datasets.data_loader import GenericDataLoader

# Template-ul URL pentru arhivele BEIR găzduite de UKP Darmstadt
url = "https://public.ukp.informatik.tu-darmstadt.de/thakur/BEIR/datasets/{}.zip"

# Descarcă arhiva setului de date și o dezarhivează în folderul "datasets"
# (înlocuiți "scifact" cu "nfcorpus" sau "fiqa" pentru celelalte seturi)
data_path = util.download_and_unzip(url.format("scifact"), "datasets")

# Încarcă cele trei structuri necesare:
# corpus – dicționar {doc_id: {"title": ..., "text": ...}}
# queries – dicționar {query_id: text_query}
# qrels – judecățile de relevanță {query_id: {doc_id: scor_relevanță}}
corpus, queries, qrels = GenericDataLoader(data_folder=data_path).load(split="test")
```

## 5. Cerințe

### Cerința 1 - Indexare

Porniți Weaviate prin Docker Compose și implementați o singură strategie de chunking (spargerea în bucăți a documentelor). Chunk-uri de dimensiune fixă cu 10% overlap este strategia recomandată chiar dacă nu este garantat că este și cea mai bună în fiecare caz. Pentru fiecare dintre cele trei seturi de date, spargeți documentele în chunk-uri, codați fiecare chunk cu modelul sentence-transformer și inserați chunk-urile împreună cu vectorii într-o colecție Weaviate separată (câte o colecție per set de date). Indexarea BM25 pe textul chunk-urilor se realizează automat în Weaviate și nu necesită cod suplimentar.

## Cerința 2 - Comparația retrieverelor pe toate cele trei seturi de date

Implementați trei funcții de query peste fiecare colecție Weaviate: una care folosește doar BM25, una care folosește doar retrieval dens (căutare vectorială) și una care folosește căutarea hibridă built-in din Weaviate, cu  $\alpha$ -ul default. Toate cele trei sunt expuse direct prin API-ul Weaviate, deci sarcina este mai mult de configurare decât de cod propriu-zis.

Rulați fiecare query de test prin fiecare retriever și calculați Recall@10, MRR și nDCG@10 raportat la qrels. Produceți un tabel comparativ cu trei rânduri (un retriever per rând) și nouă coloane (trei seturi de date  $\times$  trei metrici), sau echivalent (important este să putem analiza centralizat rezultatele).

În raport va trebui să discutați pattern-ul observat între seturile de date. Ar trebui să vedeți că nicio metodă nu domină în mod absolut pentru toate seturile de date. În raport veți argumenta, bazat și pe date, de ce stau lucrurile așa.

## Cerința 3 - RAG end-to-end pe un singur set de date

Alegeți unul dintre cele trei seturi de date — cel ale cărui rezultate de retrieval vi se par cele mai interesante — și justificați alegerea pe scurt în raport. Pentru acel set, utilizați cel mai bun retriever din Cerința 2 împreună cu Ollama și construiți un prompt de RAG format din întrebare + top-k chunk-uri regăsite + o instrucțiune de a cita ID-urile chunk-urilor folosite. Evaluați manual răspunsurile LLM-ului pe 10 query-uri din setul de test și raportați cel puțin un caz în care retrieval-ul reușește, dar LLM-ul totuși halucinează sau atribuie greșit informația, cu o discuție scurtă despre cauza posibilă.

## Cerința 4 - Calibrare prin sweep

Pe același set de date ales în Cerința 3, faceți sweep cu următorii hiperparametri: chunk size, k (din top-k),  $\alpha$  (din căutarea hibridă). Spre exemplu, puteți alege dintre dimensiunea chunk-ului (256, 512 sau 1024 tokens), valoarea top-k din retrieval (3, 5 sau 10) sau parametrul  $\alpha$  al căutării hibride (0.0, 0.25, 0.5, 0.75, 1.0), unde  $\alpha = 0$  înseamnă BM25 pur, iar  $\alpha = 1$  înseamnă dens pur.

Produceți un grafic nDCG@10 pentru valoarea testată și evaluați calitativ efectul asupra calității răspunsurilor RAG, folosind același subset de 10 query-uri din Cerința 3. Formulați o configurație recomandată, însoțită de o justificare.

## 6. Punctaj

| Criteriu                                                                                                         | Pondere |
|------------------------------------------------------------------------------------------------------------------|---------|
| Pipeline funcțional (reproductibil din README)                                                                   | 40%     |
| Comparația retrieverelor (corectitudinea metricilor, completitudinea tabelului, calitatea discuției comparative) | 25%     |
| Hyperparameter sweep (rigoare și interpretare)                                                                   | 20%     |
| Raport tehnic și video (claritate și structură)                                                                  | 15%     |

## 7. Predare

Veți încărca pe Moodle:

- raportul PDF
- arhivă cu codul sursă
- video demonstrativ

Raportul PDF de aproximativ 3-4 pagini va conține (cerințe minimale) o descriere scurtă a pipeline-ului de indexare, tabelul cu metrici, discuția comparativă între seturile de date, justificarea alegerii setului folosit pentru Cerințele 3 și 4, analiza calitativă a RAG-ului (inclusiv exemplul de halucinație) și graficul de calibrare cu configurația recomandată.

Arhiva conține codul sursă care trebuie să fie complet funcțional și reproductibil. Codul trebuie să conțină și `docker-compose.yml` pentru Weaviate, tot codul organizat în module clare, un `README.md` cu pașii de reproducere și un fișier `requirements.txt` sau `pyproject.toml`. Dacă proiectul nu este reproductibil la corectare, atunci nu va putea fi punctat.

Video-ul demonstrativ va fi o înregistrare de ecran de MAXIM două minute care demonstrează un query traversând întreg pipeline-ul — rezultate BM25, apoi rezultate dense, apoi rezultate hibride, și în final răspunsul RAG cu citări. Folosiți un encoding bun pentru video pentru a-l putea încărca pe Moodle deoarece aveți limită de upload de 50 MB. NU încărcați și datele.

## 8. Întrebări frecvente

### 1. Pot folosi un modul de vectorizare în Weaviate în loc să inserez vectorii manual?

Nu. Tema cere să vectorizați voi astfel încât totul să rămână local și reproductibil fără chei API (și apeluri către Weaviate).

### 2. Pot folosi un LLM hostat (OpenAI, Anthropic, etc.) în locul Ollama?

Nu. Tema vizează înțelegerea stack-ului local complet.

### 3. Laptop-ul meu are doar 8 GB RAM. Va funcționa?

Da. Folosind Gemma 2 2B ca LLM și MiniLM ca embedder aveți . Închideți alte aplicații în timp ce rulați sweep-ul.

### 4. Pot folosi o altă bază de date vectorială (Qdrant, Chroma, etc.)?

Nu. Weaviate este obligatoriu, deoarece căutarea hibridă built-in necesară în Task 2 tractabil în bugetul de timp dat.

### 5. Trebuie să configurez sau să tunez indexul HNSW din Weaviate?

Nu. Weaviate folosește HNSW ca index ANN default. Lăsați parametrii la valorile implicite; tunarea indexului este în afara obiectivelor temei.

### 6. Pot lucra cu un coleg?

Nu. Este o temă individuală. Puteți discuta abordări, dar codul, experimentele și raportul trebuie să fie ale voastre.